

ISOM 671

Managing Big Data

RAJIV GARG

Associate Professor

Information Systems & Operations Management

September 18, 2024

Expectations!

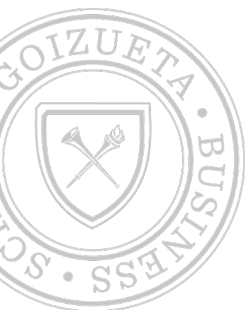
- **Stay safe and keep others safe:**
 - If you have covid like symptoms, test negative, and are attending class, please maintain distance for safety of others.
- **Discussion:**
 - Raise hand to ask any questions
 - We may have small group discussions.
- **Quiz:**
 - There may be a 5-minute quiz before class on some days, and it will be administered on Canvas (no paper). You need to make sure your canvas account is accessible during class time.
- **Be courteous:**
 - I don't mind if you are speaking to someone else or doing something else on your computer as long as you are not distracting anyone else in the class. If you are the reason for distraction, I will have to ask you to leave the class.

• **PLEASE USE NAMECARDS**



Upcoming Deadlines

▼ Assignments	
	Assignment 1: ER Model and Normalization Due Sep 12 at 1pm 30 pts
	Assignment 2: SQL Due Sep 24 at 1pm 50 pts
	Assignment 3: Loading Data in Relational Databases Due Sep 27 at 1pm 30 pts
	Group Assignment 1: Data Warehousing and CAP Theorem Due Oct 3 at 1pm 25 pts





SQL II

catchup

September 18, 2024

Text Parsing (substring nesting)

Text Analysis





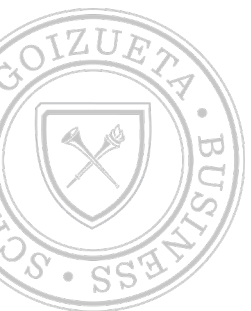
SQL III

UDF, Python, and more

September 18, 2024

SQL UDF

User Defined Functions

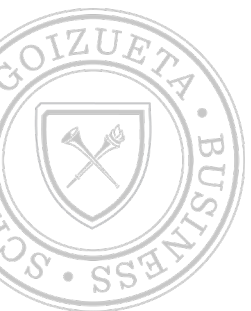


UDF – Stored Functions

- User defined functions allow simplification of repeated tasks in queries.

```
CREATE FUNCTION function_name(  
    param1,  
    param2,...  
)  
RETURNS datatype  
[NOT] DETERMINISTIC  
BEGIN  
    -- statements  
END
```

Result is “deterministic” (some outcome count received) or “not deterministic” may imply that the function can return different values for same input parameters

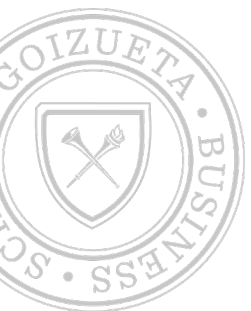


For details, please refer to:
<https://dev.mysql.com/doc/refman/8.0/en/create-procedure.html>



UDF - Hello World

- `CREATE FUNCTION hello (s CHAR(20))`
- `RETURNS CHAR(50) DETERMINISTIC`
- `RETURN CONCAT('Hello ',s,'!');`
- `SELECT hello('world');`
- `SELECT hello(first_name)`
- `FROM customer;`



For details, please refer to:

<https://dev.mysql.com/doc/refman/8.0/en/create-procedure.html>

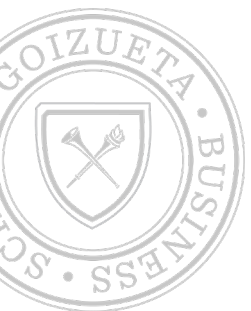
UDF – Rental Labels (\$, \$\$, \$\$\$)

- DELIMITER //
- CREATE FUNCTION RentalPriceLabel(rentalrate DECIMAL(10,2))
- RETURNS VARCHAR(20) DETERMINISTIC
- BEGIN
- DECLARE rentallabel VARCHAR(20);
- IF rentalrate < 1 THEN SET rentallabel = '\$';
- ELSEIF (rentalrate >= 1 AND rentalrate < 3) THEN SET rentallabel = '\$\$';
- ELSEIF rentalrate >= 3 THEN SET rentallabel = '\$\$\$';
- END IF;
- -- return the rental labels
- RETURN (rentallabel);
- END//
- DELIMITER ;
- SELECT title, rental_rate, RentalPriceLabel(rental_rate)
- FROM film
- ORDER BY title;



UDF – Stored Procedures

- Function is invoked to perform an action and return a value to the user/query
- You create a **stored procedure** to call an operation that is not used directly providing a return value.
- Example:
 - Function: `SELECT hello('world');`
 - Function: `SELECT hello('world'), "<- is function";`
 - Procedure: `CALL hello('world');`



UDF – Country Count

- Write a procedure that returns the number of cities in a country:

```
delimiter //
```

```
CREATE PROCEDURE citycount (IN country CHAR(3), OUT cities INT)
```

```
BEGIN
```

```
    SELECT COUNT(*) INTO cities FROM city
```

```
    WHERE country_id = country;
```

```
END //
```

```
delimiter ;
```

```
SET @count_id = (SELECT country_id from country where country="China");
```

```
CALL citycount(@count_id, @cities);
```

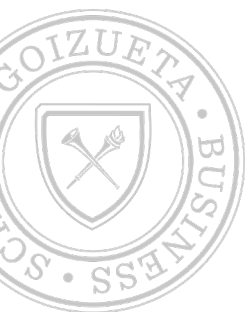
```
SELECT @count_id, @cities;
```

Input parameter

Output of
procedure/function

Temporary variable

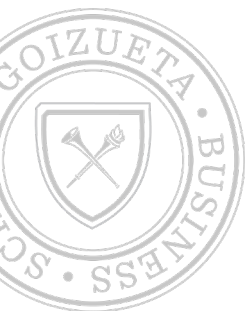
Note: *delimiter* command to change the statement delimiter from ; to // while the procedure is being defined. This enables the ; delimiter used in the procedure body to be passed through to the server rather than being interpreted by mysql itself.



SQL - Temporary Variables

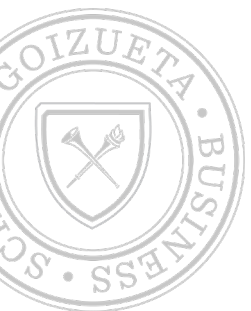
- `SET @var1 = 5;`
- `SELECT customer_id, COUNT(*) FROM rental GROUP BY customer_id HAVING COUNT(*) > @var1;`
- `SET @var2 = "ANA%";`
- `SET @mycust = (SELECT first_name from customer WHERE first_name LIKE @var2);`
- `SELECT @var1, @var2, @mycust;`

Temporary variable can contain only one value



UDF - Examples

- How would you do word count on a column in a table?
- `SELECT wordcount("THIS IS A TEST");`
- `SELECT title, wordcount(title) as wc`
- `FROM film ORDER BY wc DESC LIMIT 10;`



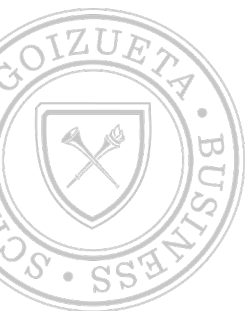
```
DELIMITER $$  
• CREATE FUNCTION wordcount(str LONGTEXT)  
    RETURNS INT  
    DETERMINISTIC  
    SQL SECURITY INVOKER  
    NO SQL  
  
    BEGIN  
        DECLARE wordCnt, idx, maxIdx INT DEFAULT 0;  
        DECLARE currChar, prevChar BOOL DEFAULT 0;  
        SET maxIdx=char_length(str);  
        SET idx = 1;  
        WHILE idx <= maxIdx DO  
            SET currChar=SUBSTRING(str, idx, 1) RLIKE '[:alnum:]';  
            IF NOT prevChar AND currChar THEN  
                SET wordCnt=wordCnt+1;  
            END IF;  
            SET prevChar=currChar;  
            SET idx=idx+1;  
        END WHILE;  
        RETURN wordCnt;  
    END  
    $$  
DELIMITER ;  
  
• SELECT wordcount("THIS IS A TEST");
```

SQL Triggers



SQL - Trigger

- A trigger is a named database object that is associated with a table, and that activates when a particular event occurs for the table. Some uses for triggers are to perform checks of values to be inserted into a table or to perform calculations on values involved in an update.
 - A trigger is defined to activate when a statement **inserts, updates, or deletes** rows in the associated table.
- Referencing rows in triggers:
 - In an INSERT trigger, only NEW.col_name can be used; there is no old row.
 - In a DELETE trigger, only OLD.col_name can be used; there is no new row.
 - In an UPDATE trigger, you can use OLD.col_name to refer to the columns of a row before it is updated and NEW.col_name to refer to the columns of the row after it is updated.



For details, please refer to:

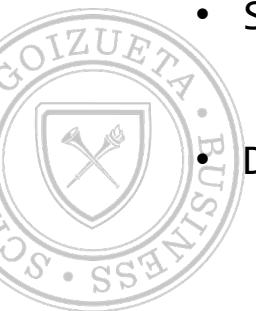
<https://dev.mysql.com/doc/refman/8.0/en/trigger-syntax.html>

SQL – Trigger (insert)

- use testDB;
- CREATE TABLE account (acct_num INT, amount DECIMAL(10,2));
- CREATE TRIGGER ins_sum BEFORE INSERT ON account
- FOR EACH ROW SET @sum = @sum + NEW.amount;
- SET @sum = 0;
- INSERT INTO account VALUES(137,14.98),(141,1937.50),(97,-100.00);
- select * from account;
- SELECT @sum AS 'Total amount inserted';
- DROP TRIGGER testDB.ins_sum;

BEFORE inserting NEW rows, add the value from NEW rows into a variable (can be a column in another table)

This will contain sum of NEW amounts added to the table.



SQL – Trigger (update)

- delimiter //
- CREATE TRIGGER upd_check BEFORE UPDATE ON account
- FOR EACH ROW
- BEGIN
- IF NEW.amount < 0 THEN
- SET NEW.amount = 0;
- ELSEIF NEW.amount > 100 THEN
- SET NEW.amount = 100;
- END IF;
- END; //
- delimiter ;

BEFORE updating with NEW data, assign a limit to top and bottom values. Could be used to avoid erroneous updates.

- SET SQL_SAFE_UPDATES = 0;
- UPDATE account SET amount = -200 WHERE account.acct_num =97;
- UPDATE account SET amount = 125 WHERE account.acct_num =137;
- select * from account;
- SET SQL_SAFE_UPDATES = 1;

Update value smaller than 0 will be changed to 0 and values greater than 100 will be changed to 100



SQL Execution Plan



SQL – EXPLAIN (index)

- EXPLAIN shows the optimal SQL execution plan
 - *find rentals that happened on May 24, 2005*

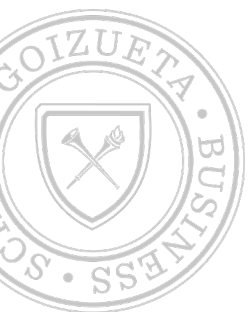
```
EXPLAIN SELECT inventory_id, customer_id  
FROM rental WHERE rental_date='2005-05-24';
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	rental	NULL	ref	rental_date	rental_date	5	const	1	100.00	Using index

Using index



For details, please refer to:
<https://dev.mysql.com/doc/refman/8.0/en/explain-output.html>



Indexing

- Easily lookup by customer name or order location
 - How many orders from Chicago?
 - How many orders by Jones?

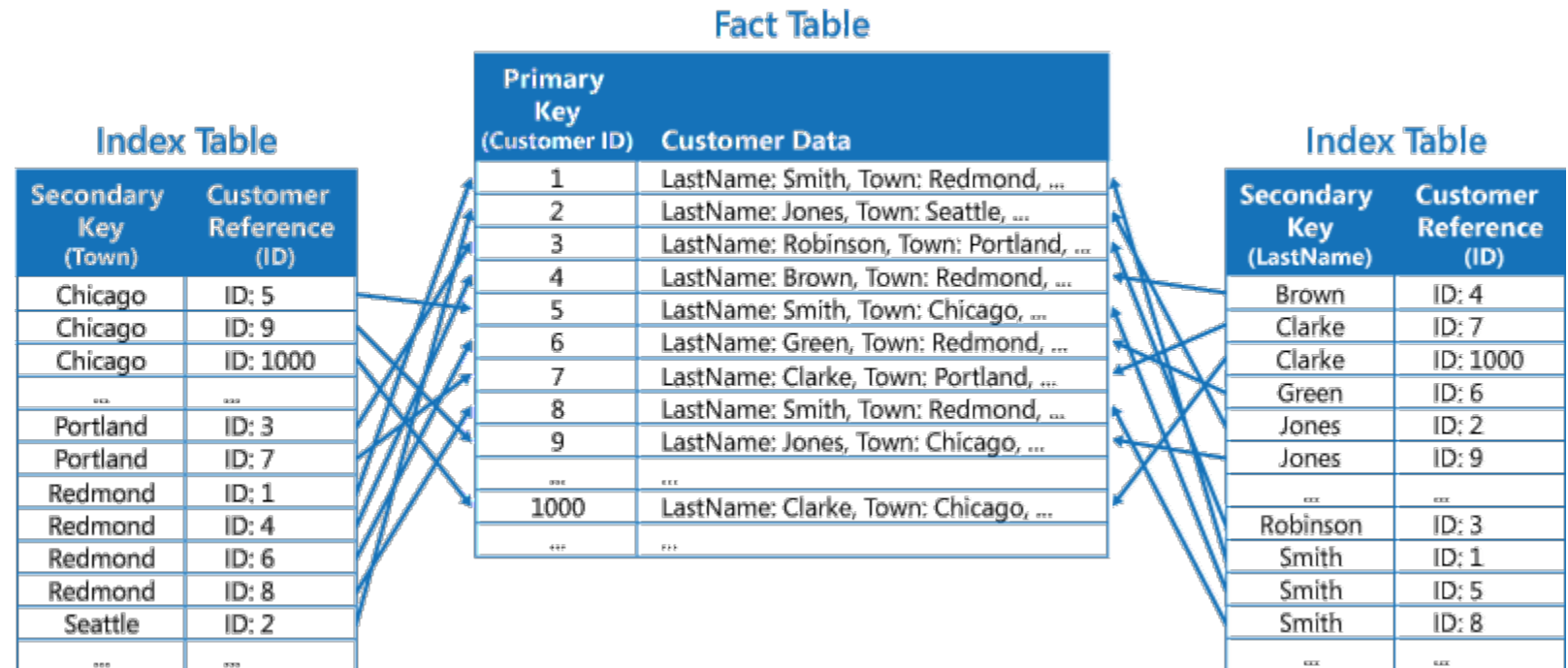


Image source: <https://docs.microsoft.com/en-us/azure/architecture/patterns/index-table>



SQL – EXPLAIN (type and rows)

- A complex query is optimized by the SQL engine
 - *find all films released in the year of 2006 where the actor with a last name of 'MIRINDA' starred*

```
EXPLAIN SELECT title, description FROM film AS f
JOIN film_actor AS fa ON f.film_id = fa.film_id
JOIN actor AS a ON fa.actor_id = a.actor_id
WHERE a.last_name = 'MIRANDA' AND f.release_year = 2006;
```

Execution order is independent of the order in query

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	a	NULL	ref	PRIMARY,idx_actor_last_name	idx_actor_last_name	137	const	1	100.00	Using index
1	SIMPLE	fa	NULL	ref	PRIMARY,idx_fk_film_id	PRIMARY	2	sakila.a.actor_id	27	100.00	Using index
1	SIMPLE	f	NULL	eq_ref	PRIMARY	PRIMARY	2	sakila.fa.film_id	1	10.00	Using where

rows will be accessed using an index, one row is read from the table for each combination of rows from the previous tables

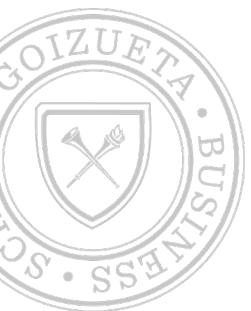
Number of rows processed

Using 10% rows



SQL – EXPLAIN (select type)

select_type Value	JSON Name	Meaning
SIMPLE	None	Simple SELECT (not using UNION or subqueries)
PRIMARY	None	Outermost SELECT
UNION	None	Second or later SELECT statement in a UNION
DEPENDENT UNION	dependent (true)	Second or later SELECT statement in a UNION , dependent on outer query
UNION RESULT	union_result	Result of a UNION .
SUBQUERY	None	First SELECT in subquery

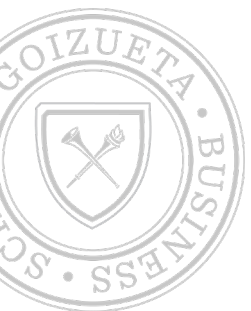


For details, please refer to:
<https://dev.mysql.com/doc/refman/8.0/en/explain-output.html>

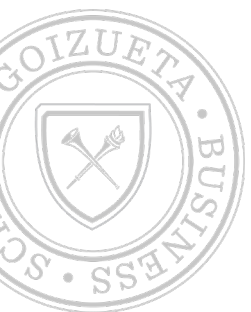
SQL - EXPLAIN

- To optimize queries:
 - **Index check:** Make sure number of rows processed are few and tables are indexed when a column in those tables is likely to be used frequently
 - **Lean data:** When possible, avoid redundant data selection in query compounding (JOIN, WHERE, SUBQUERIES, etc.)
 - EXPLAIN SELECT min(rental_rate) from film;
 - vs
 - EXPLAIN SELECT min(rental_rate) from film WHERE rental_rate>0;

33% of data is filtered



SQL & Python



SQL & Python

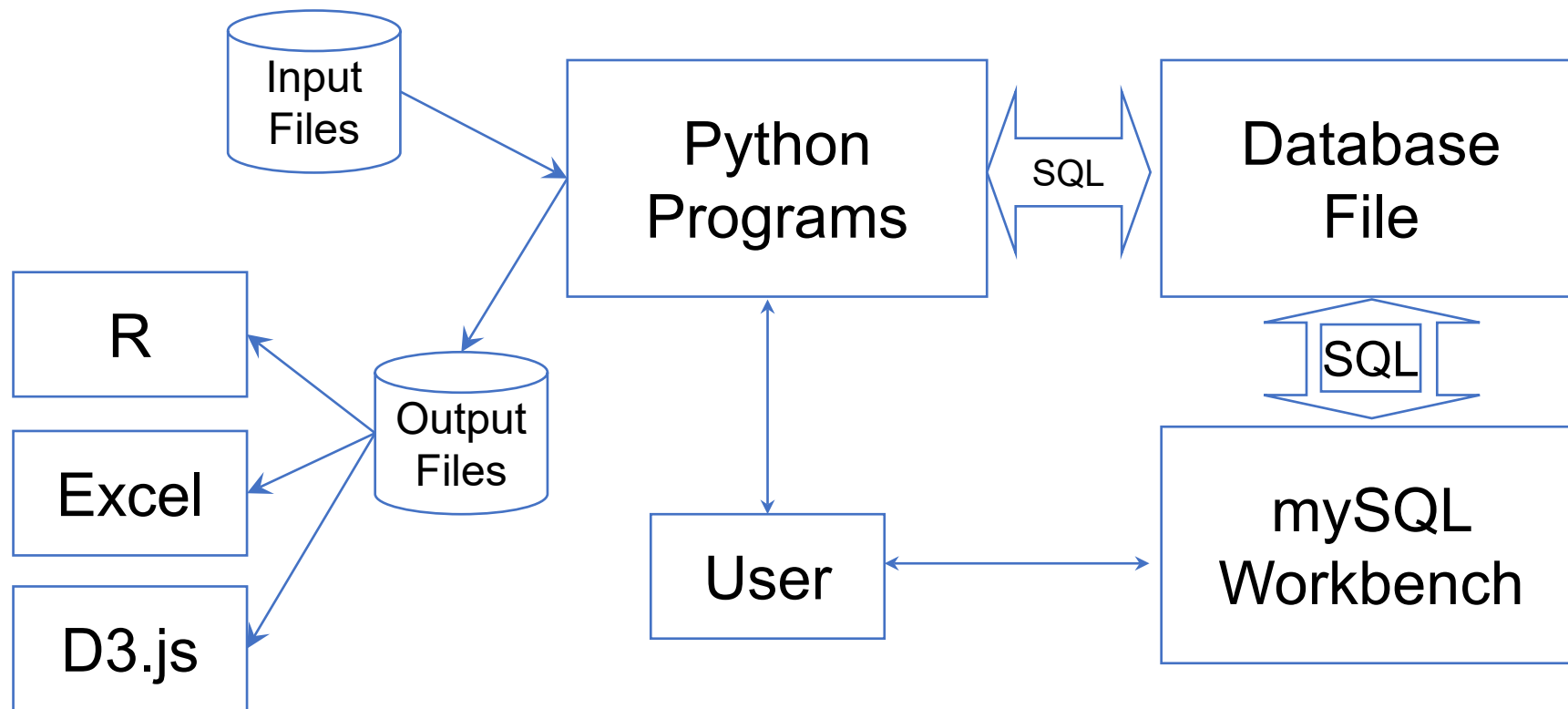


Image source: Charles Severance (py4e.com)



Accessing Databases

- Install: <https://pypi.org/project/pyodbc/>
 - `pip install pyodbc`
- Reads and writes MS Access database, MySQL, and many more
- ODBC: Open Database Connectivity
 - Microsoft introduced the ODBC standard in 1992.
 - ODBC is a specification for a database API that is independent of any one DBMS or operating system.
 - “The functions in the ODBC API are implemented by developers of DBMS-specific drivers. Applications call the functions in these drivers to access data in a DBMS-independent manner.”
 - <https://docs.microsoft.com/en-us/sql/odbc/reference/what-is-odbc>

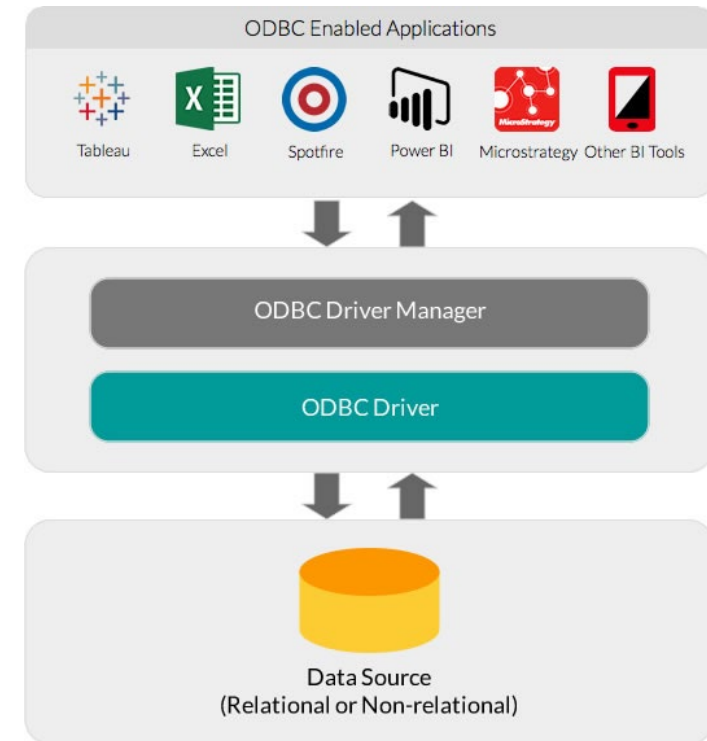
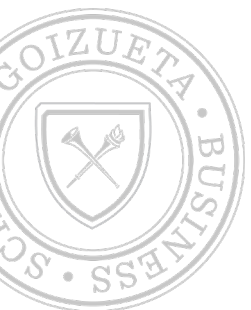


Image source: <https://www.simba.com/resources/odbc/>



ODBC vs MySQL Connector

- `pip install pyodbc`
 - Works for all database software installed on your computer (including MS Access)
 - Works for same bit version of Python (x86) and RDBMS (x86)
 - Or you need the database specific connector
- `pip install mysql-connector-python`
 - Specific to MySQL
 - Works when you have different version of Python (x86) and MySQL (x64)



Reading MySQL Data

- Using ODBC connection to read databases

```
import mysql.connector
```

```
mydb = mysql.connector.connect(  
    host="localhost",  
    user="root",  
    password=open("mysql_pass.txt", "r").readline() #SAVE PASSWORD IN A FILE  
)
```

Mysql connector

```
cursor = mydb.cursor()  
cursor.execute('select * from products')
```

```
for row in cursor.fetchall():  
    print (row)
```

```
(1, 'apple', 3.5, 4.99, 100.0)  
(2, 'orange', 0.87, 1.49, 200.0)  
(3, 'milk', 2.68, 3.99, 300.0)  
(4, 'bread', 1.41, 1.99, 400.0)  
(5, 'beer', 9.9, 19.99, 100.0)  
(6, 'diaper', 18.21, 29.99, 50.0)
```

SQL query to execute



Reading MySQL Data

- `import mysql.connector`
- `mydb = mysql.connector.connect(`
- `host="localhost",`
- `user="root",`
- `password=open("mysql_pass.txt", "r").readline()`
- `)`
- `mycursor = mydb.cursor()`
- `mycursor.execute("SHOW databases")`
- `print("AVAILABLE DATABASES:\n", mycursor.fetchall())`
- `mycursor.execute("USE sakila")`
- `mycursor.execute("SELECT * FROM customer")`
- `myresult = mycursor.fetchall()`
- `print("RESPONSE TO QUERY:")`
- `for x in myresult:`
- `print(x)`

Open a connection
to MySQL server

Protecting your
password 😊

Show available
databases

Query database and
fetch results

print results



Writing Query output to CSV

```
mycursor.execute("SELECT * FROM customers")  
myresult = mycursor.fetchall()
```

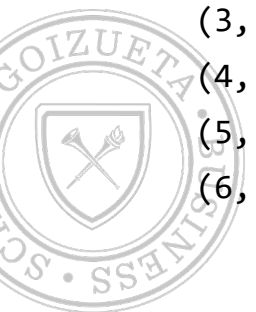
```
fname = "query_out.csv"  
fhandle = open(fname, "w", newline="")  
csvfile = csv.writer(fhandle)  
csvfile.writerows(myresult)  
fhandle.close()
```

```
(1, 'apple', 3.5, 4.99, 100.0)  
(2, 'orange', 0.87, 1.49, 200.0)  
(3, 'milk', 2.68, 3.99, 300.0)  
(4, 'bread', 1.41, 1.99, 400.0)  
(5, 'beer', 9.9, 19.99, 100.0)  
(6, 'diaper', 18.21, 29.99, 50.0)
```

query data

Save to csv

apple3.50	4.99	922
orange0.87	1.49	432
milk2.68	3.99	761
bread1.41	1.99	930
beer9.90	19.99	883
diaper18.21	29.99	364



Example - JOIN

- Extract shipping addresses for all customers and save the following columns:
first name, last name, address, city, state, zip

```
SELECT first_name, last_name, line1, city, state, zip_code
FROM customers c JOIN addresses a
  ON c.customer_id = a.customer_id
 AND c.shipping_address_id = a.address_id
```

```
query = 'SELECT first_name, last_name, line1, city, state, zip_code '\
        'FROM customers c JOIN addresses a '\
        'ON c.customer_id = a.customer_id '\
        'AND c.shipping_address_id = a.address_id'
getData(query)
```

\: allows string to be broken over multiple lines

Remember to leave a space at the end, else it will join the last word with the first word in next line



Example – MULTI-JOIN

- Extract customer name, item name ordered, item price, item discount and quantity from tables: orders, customers, order_items, and products

```
SELECT last_name, first_name, order_date, product_name, item_price, discount_amount, quantity
FROM customers c
  JOIN orders o ON c.customer_id = o.customer_id
  JOIN order_items oi ON o.order_id = oi.order_id
  JOIN products p ON oi.product_id = p.product_id
ORDER BY last_name, order_date, product_name
```

```
query = 'SELECT last_name, first_name, order_date, product_name, item_price, discount_amount,
quantity '\
        'FROM customers c '\
        'JOIN orders o ON c.customer_id = o.customer_id '\
        'JOIN order_items oi ON o.order_id = oi.order_id '\
        'JOIN products p ON oi.product_id = p.product_id '\
        'ORDER BY last_name, order_date, product_name'
getData(query)
```



Example – SUBQUERIES

- Extract which products have a list price that's greater than the average list price for all products?

```
SELECT product_name, list_price
FROM products p
WHERE list_price > (SELECT AVG(list_price) FROM products)
ORDER BY list_price;
```

```
query = 'SELECT product_name, list_price '\
'FROM products p '\
'WHERE list_price > (SELECT AVG(list_price) FROM products) '\
'ORDER BY list_price;'
getData(query)
```

Additional aggregation functions:
<https://dev.mysql.com/doc/refman/8.0/en/aggregate-functions.html>



Example – Data Manipulation

- Show the last 4 digits of card number used in orders and mask all previous digits with “X”

```
SELECT card_number,  
       LENGTH(card_number) AS card_number_length,  
       SUBSTRING(card_number, LENGTH(card_number)-3) AS last_four_digits,  
       CONCAT('XXXX-XXXX-XXXX-', SUBSTRING(card_number, LENGTH(card_number)-3)) AS  
formatted_number  
FROM orders;
```

```
query = 'SELECT card_number, '\  
        'LENGTH(card_number) AS card_number_length, '\  
        'SUBSTRING(card_number, LENGTH(card_number)-3) AS last_four_digits, '\  
        'CONCAT("XXXX-XXXX-XXXX-", SUBSTRING(card_number, LENGTH(card_number)-3)) AS  
formatted_number '\  
        'FROM orders;'  
getData(query)
```

Pass quotes as double quotes



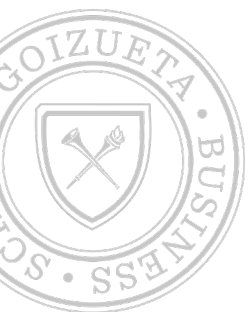
SQL in Python

- SQL + Python/Pandas + ML = “Master of the Data Science”
- Why?
 - Bigger: Import/export/manipulate/analyze all kinds of data quickly
 - Faster: Use SQL engine for faster data process and storage (you can even connect to Spark, Hive, etc)
 - Cheaper: Massive volume of open-source libraries for data analysis and ODBC drivers are usually included for free!



SQL & Python

Google Cloud



Reflect

Close your eyes, relax,
and reflect on ONE take
away from today's class.

5:00





**“SQL + Python + ML =
‘Master of Data Science’”**